

Machine Learning Project

Italian Rap Songs Dataset

Filippo Appetito
email filippo.appetito@sns.it

Ludovico Lenzuni
email ludovico.lenzuni@sns.it

May 2026

1 Dataset and Preparation

1.1 Data Loading and Merging

As first thing we imported the `tracks.csv` and `artists.csv` and printed the heads to observe their nature. We then observed that they contain 11166 tracks and 104 artists.

Before merging the two datasets we added prefixes "track_" and "artist_" to the columns names so to simplify the distinction between artist and song level features. Then we merged the two datasets using as matching variables "track_id_artist" and "artist_id_author". The final dataset contained 11166 non-null entries in "artist_name" from which we can infer that the matching was successful for all the input tracks.

From printing the dataset info we noticed that all the columns have at least ≈ 6000 non-null entries except for the "artist_active_end" one which has 0. We decided to drop it as it was a useless (and meaningless) feature.

1.2 Cleaning and Deduplication

Before proceeding with the analysis, we addressed several data quality issues, including duplicated tracks and inconsistencies in artist names.

We faced the duplicated tracks problem. We noticed that there are 73 songs with the same "track_id". However printing them we noticed that most of them were different songs with same id. Printing songs with same "track_id" and "track_title" we spotted two pairs of exactly duplicated songs. We dropped the two extras. We also checked for inconsistencies in the authors names. We printed all the sets of different "track_primary_artist" corresponding to the same "artist_name". We found all plausible pseudonyms corresponding to the same person (e.g. `highsnob`, `mike24`).

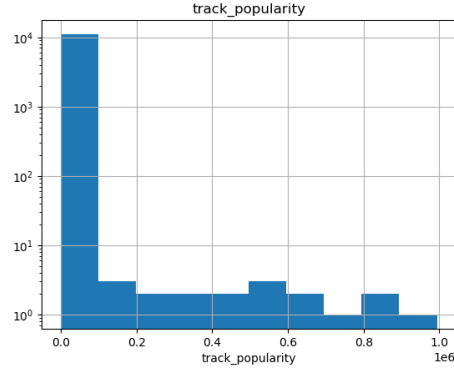


Figure 1: Histogram for the track_popularity feature

1.3 Outlier Analysis and Feature Engineering

Our first approach to spot extreme values was to use standard deviation. Supposing that the numerical features are normally distributed, we can compute that the probability that a single event lies further than 5σ from the mean value is $< 10^{-6}$. So on $\approx 10^4$ entries we are very unlikely to find even one of such. However, printing the number of these "outliers" per feature we see that the majority of number features has at least 10, and many have hundreds of outliers. So, normal distribution is not a good distribution for many variables.

We then switched to the "track_year" variable which was giving problems in the numeric conversions. Printing its unique values we noticed that many were absurd or even string. We then filtered the good ones imposing the condition "artist_active_year" (feature extrapolated from "artist_active_start") $<$ "track_year" $<$ "2027". This coincidentally left with just ≈ 5000 not Nan possibly eliminating some correct years that had no "artist_active_year" thus failing the condition. However in this way we are sure that all years are plausible.

To better visualize the distribution of some quantities we chose to print the histograms of the suggested variables. For the swear words we chose to do a cumulative plot highlighting in two different colors the two contributions. Thanks to the log-scale chosen for the y-axis we were able to see single events much farther from the bulk in the popularity, duration, bpm and pageviews distributions.

At this stage we defined four new variables:

- $\text{track_bumpiness} = (\text{track_spectral_complexity}) / (\text{track_rolloff} - \text{track_centroid})$
- $\text{track_avg_swear_words} = \text{track_tot_swear_words} / \text{track_n_tokens}$
- $\text{track_artist_age} = \text{track_year} - \text{artist_birth_year}$
- $\text{track_tokens_per_sec} = (1000 * \text{track_n_tokens}) / \text{track_duration_ms}$

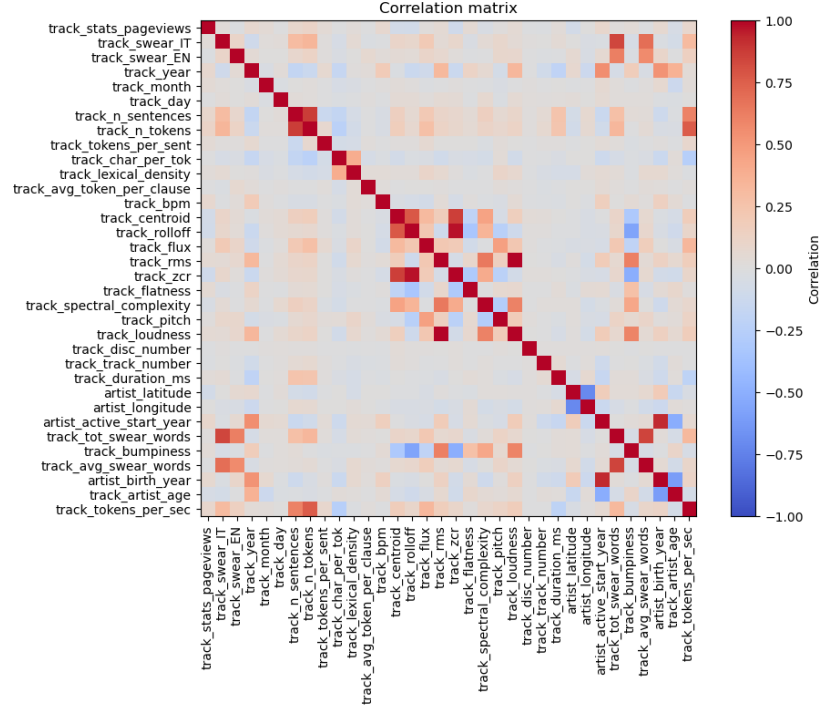


Figure 2: Heat map of the correlation between different features

The first one is an esteem of the density of peaks in the central part of the spectrum. The second it the average number of swear word per token. The third is an esteem of the artist age when he/she published the song. The fourth is the average number of tokens per seconds, estimating how fast the artist sings.

1.4 Correlation Analysis

Having defined these new variables, we printed the correlation matrix.

We then printed a list with the most (positively or negatively) correlated pairs: We see that many spectral features (rolloff, zcr, centroid) are quite correlated. Analogously for volume ones (loudness and rms). The active start obviously is correlated to the birth year. Also we noted that the total number of swear words is correlated to both the italian and the english ones.

For the negative ones we notice that latitude and longitude are negatively correlated due to the geographical tilt of the italian peninsula. Also from the negative correlation between birth year and track artist age we deduce that new artist usually starts singing earlier than old ones.

The engineered features will be used in the pattern mining analysis.

Feature 1	Feature 2	Corr
track_rms	track_loudness	0.995555
track_rolloff	track_zcr	0.969097
artist_active_start_year	artist_birth_year	0.927140
track_n_sentences	track_n_tokens	0.867736
track_centroid	track_zcr	0.864452
track_tot_swear_words	track_avg_swear_words	0.850804
track_swear_IT	track_tot_swear_words	0.842228
track_centroid	track_rolloff	0.775643
track_n_tokens	track_tokens_per_sec	0.760372
track_swear_IT	track_avg_swear_words	0.693247
track_rms	track_spectral_complexity	0.644292
track_swear_EN	track_tot_swear_words	0.623791
track_rms	track_bumpiness	0.616917
track_n_sentences	track_tokens_per_sec	0.609353
track_spectral_complexity	track_loudness	0.608888
track_loudness	track_bumpiness	0.599506
track_swear_EN	track_avg_swear_words	0.564019
track_year	artist_active_start_year	0.551709
artist_birth_year	artist_latitude	0.525355
track_zcr	track_bumpiness	-0.503794
artist_active_start_year	track_artist_age	-0.511536
track_rolloff	track_bumpiness	-0.569447
artist_birth_year	track_artist_age	-0.603261
artist_latitude	artist_longitude	-0.704038

2 Pattern Mining

2.1 Data discretization

To discretize in a meaningful way our selected variables, we have researched the typical ranges of the quantities we want to bin, and, for different variables, we have followed some heuristic rules:

- BPM: cut at 90 and 120 BPM, common musicology thresholds for slow/medium/fast tempos.
- Popularity: use 0–30–70–100 to map to low/medium/high on a 0–100 scale.
- Duration: short < 3 min, medium 3–5 min, long > 5 min (typical radio lengths).
- Swear density: compute swear words per minute; split low/high by median to avoid skew from heavy tails.

Following a similar reasoning, but also keeping in mind the range of values presented to discretize the variables in a meaningful way, we have:

- Lexical density: split between "Low" and "High" at a value of 0.5, which is around the median. The range is also between 0 and 1.
- Loudness: has a range between 0 and about 80, so we labeled the ranges 0–27 as "Soft", 27–54 as "Medium", and >54 as "Loud". The distribution is negatively skewed, so most of the entrances will be classified as Soft or Medium.
- Acuteness: the itemization of the "centroid" variable. The variable follows a gaussian-like distribution in the range 0.0 - 0.3, so we itemized it as "Low" or "High" for values lower or higher than 0.15, respectively.

2.2 Pattern mining

We encoded all the tracks as transactions, and used the Apriori algorithm to find the frequent itemsets.

We chose a minimum support of 0.2. With this choice, 75 itemsets remain.

To find the meaningful association rules, we imposed a minimum confidence of 0.6, and then ordered the results by the lift. Here are the 10 most relevant association rules:

Antecedents	Consequents	Support	Confidence	Lift	Leverage
(acuteness_Low, popularity_Low)	(loudness_Soft)	0.214727	0.682906	1.119260	0.022880
(swear_density_High)	(lexical_density_High)	0.331273	0.667268	1.114410	0.034010
(swear_density_High, acuteness_Low)	(lexical_density_High)	0.209263	0.656549	1.096507	0.018418
(swear_density_Low, acuteness_Low)	(loudness_Soft)	0.239900	0.667498	1.094006	0.020614
(duration_Short)	(lexical_density_High)	0.217773	0.653319	1.091113	0.018185
(loudness_Soft, swear_density_Low)	(acuteness_Low)	0.239900	0.740597	1.083524	0.018493
(swear_density_Low, duration_Medium)	(loudness_Soft)	0.208457	0.658834	1.079806	0.015407
(loudness_Medium)	(lexical_density_High)	0.245275	0.640318	1.069400	0.015917
(swear_density_Low)	(loudness_Soft)	0.323927	0.652354	1.069187	0.020961
(lexical_density_Low)	(duration_Medium)	0.261489	0.662957	1.068060	0.016663

2.3 Binary features

Given the most relevant association rules, we defined four new binary features:

- “obtuse_unpopular”: “acuteness_Low” AND “popularity_Low”
- “polite_medium”: “swear_density_Low” AND “duration_Medium”
- “soft_popular”: “loudness_Soft” AND “popularity_High”
- “polite_soft”: “swear_density_Low” AND “loudness_Soft”

3 Clustering

3.1 Feature selection

Out of all the feature, be them original or engineered, we extrapolated a relevant subset, grouping them by category. We differentiated four macro features:

- Audio: “centroid”, “rolloff”, “pitch”
- Textual: “tokens_per_sent”, “lexical_density”, “avg_swear_words”
- Popularity: “popularity”, “stats_pageviews”
- Geography: “latitude”, “longitude”

We grouped the attributes chosen for every category not only for conceptual resemblance, but we also tried to be coherent in the “units of measurement” of every group, so that we could start a first clustering attempt without the normalization. It will turn out that the results with and without the normalization are not significantly different.

We also treated the “geography” group differently from the others, since it really benefits from the absence of normalization, and it has a semi-discrete structure, since every different track by the same singer contributes to the same

point on the geographical coordinates.

This leads to a kind of discretization of the whole data if analyzed alongside the rest of the groups, therefore forming false clusters.

Since we want to look for cluster that span different groups, we need to normalize every entry and set the mean to zero.

3.2 Algorithm

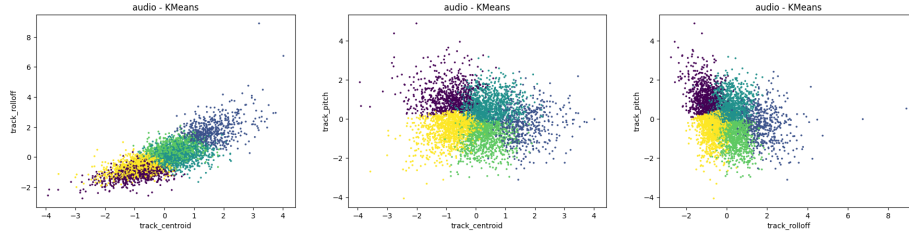
We employed the KMeans and DBSCAN clustering algorithms together with the silhouette and Davies-Bouldwin score to evaluate in a single shot the best number of clusters and epsilon, for each algorithm respectively.

Since the silhouette score needs to be maximized and the DB score needs to be minimized, the metric we chose to maximized to choose the optimal parameters is the difference of the two, since we checked that their respective ranges do not differ enough to form a considerable bias for one of the two metrics.

3.3 Kmeans

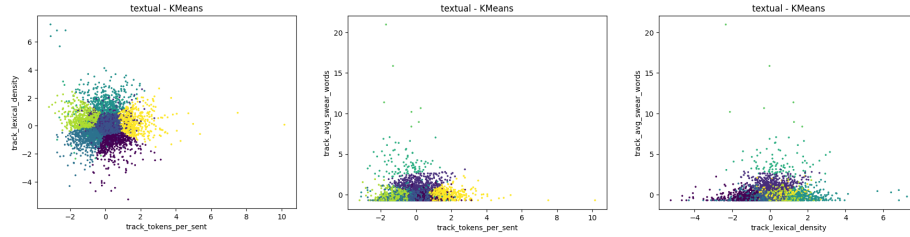
We decided to first cluster only inside the group, for every group. This are the results:

Audio



Clusters: 5, Silhouette: 0.28, DB score: 1.0

Textual



Clusters: 9, Silhouette: 0.24, DB score: 1.0

Popularity

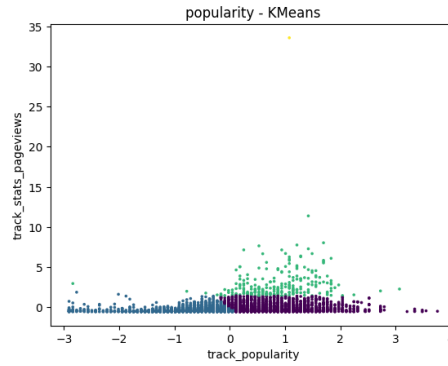
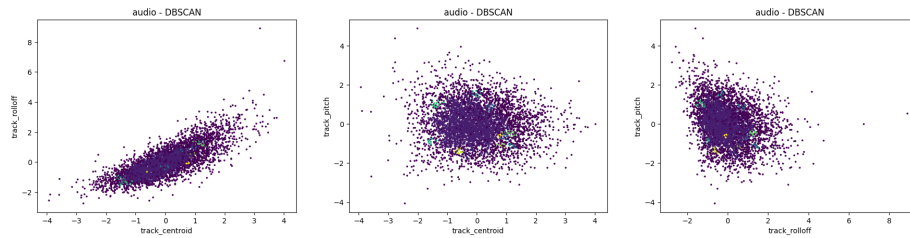


Figure 3: Clusters: 4, Silhouette: 0.46, DB score: 0.58

3.4 DBSCAN

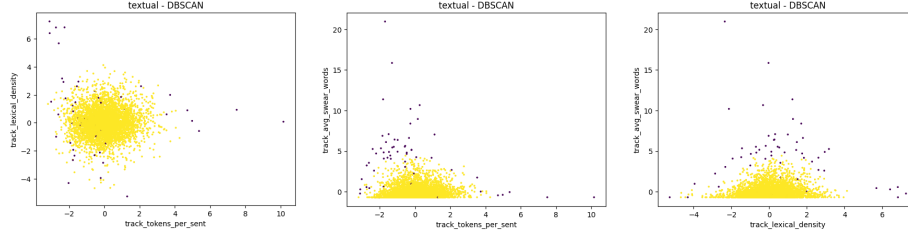
We did the exact same thing for DBSCAN, obtaining the best epsilon and subsequently the number of clusters:

Audio



Clusters: 13, Epsilon: 0.2, Silhouette: -2.4, DB score: 1.9

Textual



Clusters: 2, Epsilon: 1.0, Silhouette: -0.61, DB score: 1.3

Popularity

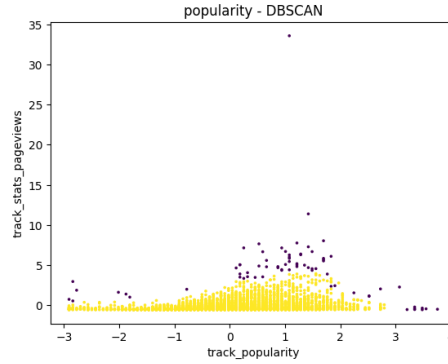


Figure 4: Clusters: 2, Epsilon: 0.4, Silhouette: -0.091, DB score: 0.81

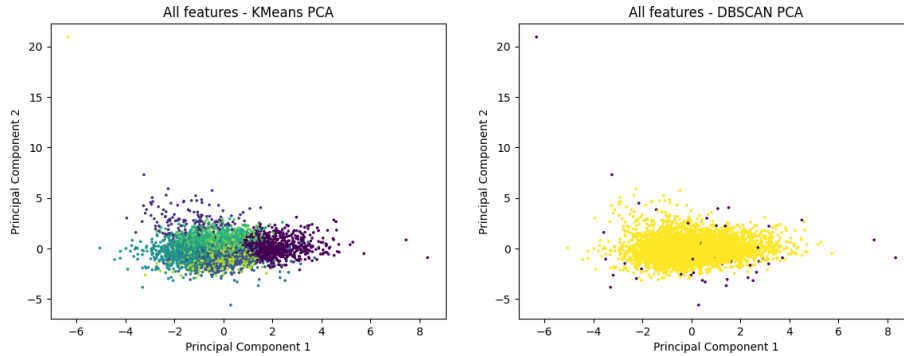
3.5 Dimensionality reduction

In the end, we joined all the three groups together (since they are normalized), and performed again the KMeans and DBSCAN clustering. We obtained:

KMeans Clusters: 9, Silhouette: 0.12, DB score: 1.4

DBSCAN Clusters: 2, Epsilon: 2.4, Silhouette: -1.74, DB score: 2.1

Immediately after, we applied the PCA algorithm to reduce the high dimensionality and obtained the following plots:



3.6 Conclusions

After this clustering analysis, we see that the data does not present any particular clustered structure.

Most of the variables are concentrated mostly around the mean, which hints at the absence of clusters. This also means that the DBSCAN algorithms will either form one big cluster with the remaining points farthest from the mean labeled as noise, as it happened in the textual and popularity groups, or, as seen in the audio clustering, form anomalous small clusters in the regions of medium density.

The analysis of the clustering of all the selected variables, with the assist of PCA, leads us to analogous observations and conclusions.

4 Classification

4.1 Data Preparation

After repeating the data processing of the data preparation and pattern mining parts, we tried to see if there were some missing "artist_region" entries that we could fill. With a newly defined function we found that some rows had empty region but "Ancona" in the province column. We then added "Marche" in the region parameter for those tracks.

Then we analyzed the artist names for rows with missing artist region. Many of them had an Italian birth place retrievable through Google search and we added the region. Some were born in a foreign country or groups of people coming from different regions. We dropped those rows.

We created the feature "artist_macro_region" as suggested, noticing a high abundance (54,5 %) of "North" entries, due to the Milan rap scene.

We then prepared our dataset for the classification with 16 features (all numerical except the macro-region) all with 10722 entries. We picked just 16 columns, choosing just one representative in each highly correlated pair.

We then printed the scatter plot for the variable pairs choosing different colors for the five macro-regions. That were no trivial plots (like almost lines or well

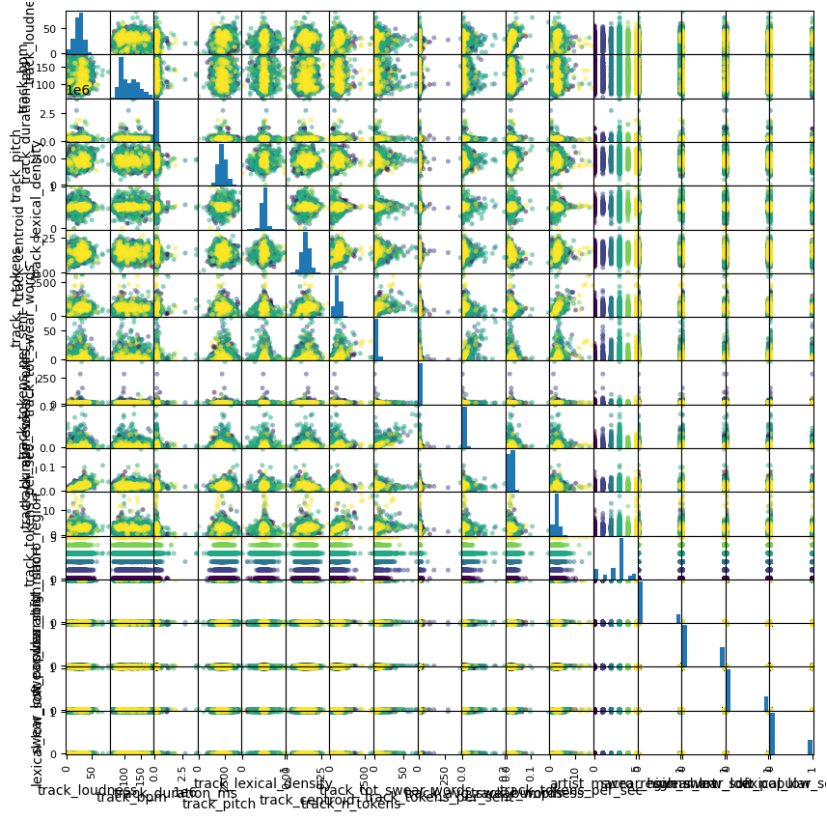


Figure 5: Scatter plot of the numerical and boolean features selected as predictors for the classification task

separated color clusters).

We then proceeded with the preparation of the test and training set. We stratified for macro region between the two subsets and choose a proportion of 70 % training and 30 % for test.

4.2 Decision Trees

We trained a simple decision tree from sklearn. Changing the parameter we spotted that the best choice for the parameters was `max_depth=3`. This had an accuracy of 54,2 %. From the confusion matrix we confirmed that the tree was simply assigning every test entry as "North". This maybe is the highest accuracy it could reach on the training set but was not very helpful. Indeed every macro-region except the "North" had f1-score=0.

To overcome this issue, we trained a second decision tree but with the parameter `class_weight='balanced'` in order to exploit more the less populated classes. We

Table 1: Confusion Matrix of the first decision tree)

True\Predicted	Campania	Center	Lazio	North	Sardegna	South
Campania	0	0	0	438	0	0
Center	0	0	0	185	0	0
Lazio	0	0	0	462	0	0
North	0	0	0	1744	1	0
Sardegna	0	0	0	166	0	0
South	0	0	0	222	0	0

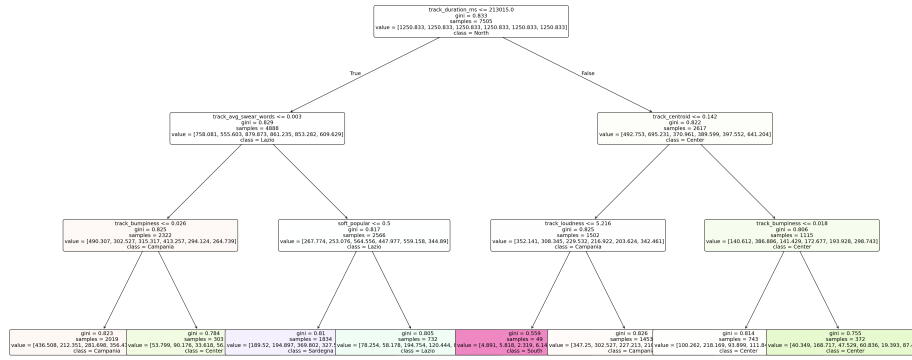


Figure 6: Second Decision tree scheme with (criterion='gini', max_depth=3, class_weight='balanced', min_samples_split=2, min_samples_leaf=1)

ended up with an accuracy of just 14,5 % but non-null f1-scores except for the "North" class. We concluded that this was a more insightful classification and we report the schematic representation.

Printing the feature importance for this second tree we noticed that some had 0.0:

- track_duration_ms 0.3127266163002929
- track_avg_swear_words 0.20825867330326403
- track_bumpiness 0.15963962133823772
- soft_popular 0.11404853476741196
- track_centroid 0.10288049892264606
- track_loudness 0.10244605536814742
- track_bpm 0.0

Table 2: Confusion matrix for the second decision tree

True\Predicted	Campania	Center	Lazio	North	Sardegna	South
Campania	270	57	34	0	74	3
Center	78	62	15	0	30	0
Lazio	188	77	71	0	123	3
North	784	323	168	0	465	5
Sardegna	67	24	15	0	59	1
South	96	68	13	0	38	6

- track_pitch 0.0
- track_lexical_density 0.0
- track_n_tokens 0.0
- track_tot_swear_words 0.0
- track_tokens_per_sent 0.0
- track_tokens_per_sec 0.0
- obtuse_unpopular 0.0
- polite_medium 0.0
- polite_soft 0.0

We noticed that two of the engineered features (avg_swear_words, bumpiness and soft_popular) have a non-null importance.

We thus trained a third simple decision tree on a dataset with the 7 most important features. However the third tree had a worse accuracy (13.3 %).

4.3

To improve upon the decision tree results, we trained a Random Forest classifier and a Support Vector Classifier (SVC). For the Random Forest, we used the following parameters: n_estimators=100, max_depth=3, min_samples_split=5, min_samples_leaf=4, criterion='gini', class_weight='balanced', random_state=42. As with the second decision tree, we balanced the class weights to avoid the model collapsing onto the dominant "North" class. The resulting accuracy was 15.2 %, consistent with the balanced decision tree, and the classification report (Table 3) confirms non-zero f1-scores across most regions, with the notable exception of "North", which the model rarely predicts due to the balancing constraint. For the SVC, the class_weight='balanced' option is not available. As a consequence, the model behaved analogously to the first unbalanced decision tree, achieving an accuracy of 54.2% but assigning virtually all test entries to the "North" class, making it uninformative for the minority regions. Overall, all three classifiers confirm the same pattern: without class balancing,

Table 3: classification report for the Random Forest classification

macro-region	precision	recall	f1-score	support
Campania	0.21	0.43	0.28	438
Center	0.10	0.33	0.16	185
Lazio	0.18	0.25	0.21	462
North	0.62	0.01	0.03	1745
Sardegna	0.08	0.36	0.13	166
South	0.11	0.14	0.12	221
accuracy			0.15	3217
macro avg	0.22	0.25	0.15	3217
weighted avg	0.41	0.15	0.11	3217

models default to predicting "North"; with balancing, accuracy drops but the classification becomes more informative across all macro-regions. This suggests that the geographic signal in the audio and textual features is weak and roughly uniform across regions.

5 Conclusions

In this work we analyzed a dataset of Italian rap songs, combining track-level audio and textual features with artist-level metadata. After a careful data preparation phase, which included merging, deduplication, outlier handling, and feature engineering, we obtained a clean dataset of approximately 11000 entries. Pattern mining via the Apriori algorithm revealed some interesting associations. The most significant rules, ranked by lift, suggest that songs with high swear density tend to also have high lexical density, and that soft, low-acuteness tracks are more likely to have low swear density. While the lift values are modest (all below 1.12), the rules are consistent and could be interpretable within the context of Italian rap music.

However, a low lift means that we should be very careful in making direct interpretations, since the correlations could be more subtle and intricate.

Clustering analysis via both KMeans and DBSCAN showed that the data does not present a clearly separated cluster structure. Most variables are concentrated around the mean, leading to low silhouette scores across all feature groups. The only partial exception is the popularity group, where KMeans achieved a silhouette score of 0.46, suggesting a mild separation between popular and unpopular tracks. PCA confirmed the absence of strong geometric structure in the full feature space.

Furthermore, the density of the data is very non-uniform, leading both algorithms to struggle a little bit in extracting meaningful results.

Classification of tracks by artist macro-region proved to be a challenging task. A simple decision tree with default parameters collapsed to always predicting

"North", reflecting the dominance of the Milan rap scene (54.5% of entries). Introducing balanced class weights produced a more informative model, with non-zero f1-scores across most regions, at the cost of overall accuracy. The Random Forest classifier confirmed this trade-off. The difficulty of the task suggests that audio and textual features alone carry limited geographic signal, and that region of origin is not strongly encoded in the musical or lyrical style of the tracks.